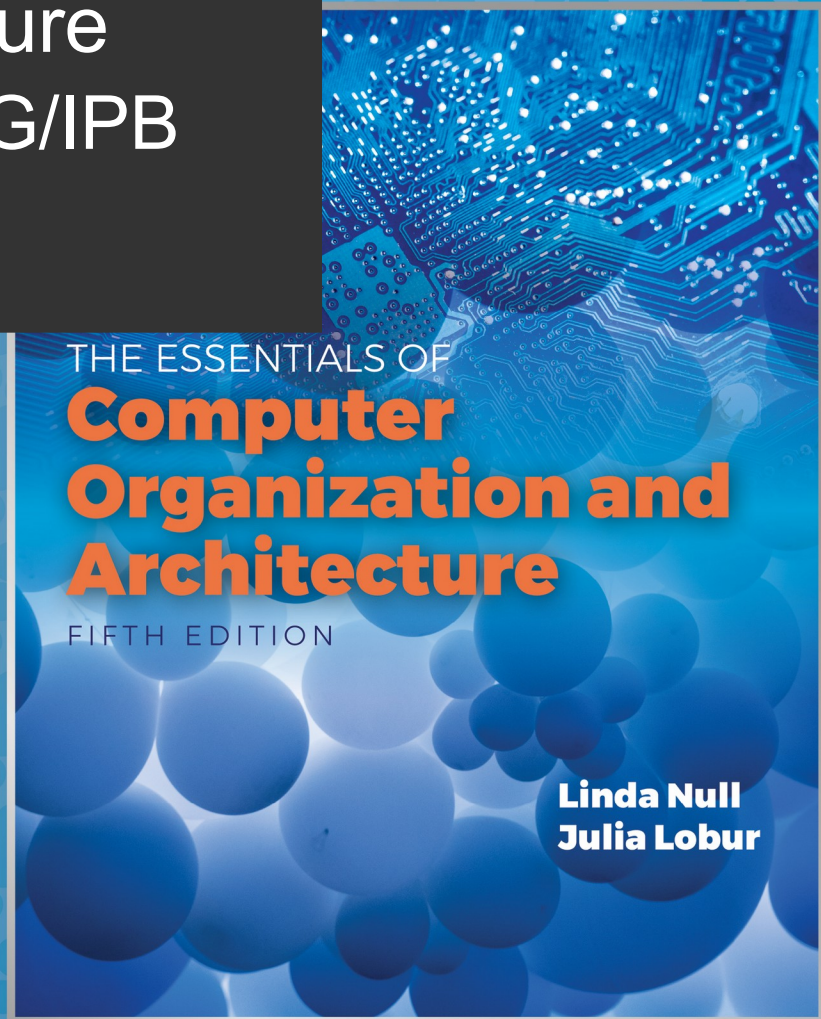


Computer Architecture Informatics Eng., ESTiG/IPB 2025/2026

José Rufino

(mainly based on Chapter 4 of the book “The Essentials of Computer Organization and Architecture” of Linda Null & Julia Lobur)

Unit 3: **MARIE** **Architecture**



Objectives

- Learn the basic components of a von Neumann CPU architecture and the basic organization of Main Memory
- Be able to explain how each CPU component and how the Main Memory, together, contribute to program execution
- Understand a simple architecture built around these basic concepts, and build programs that target such architecture
- Know how machine code is generated from assembly programs, and how machine instructions are decoded

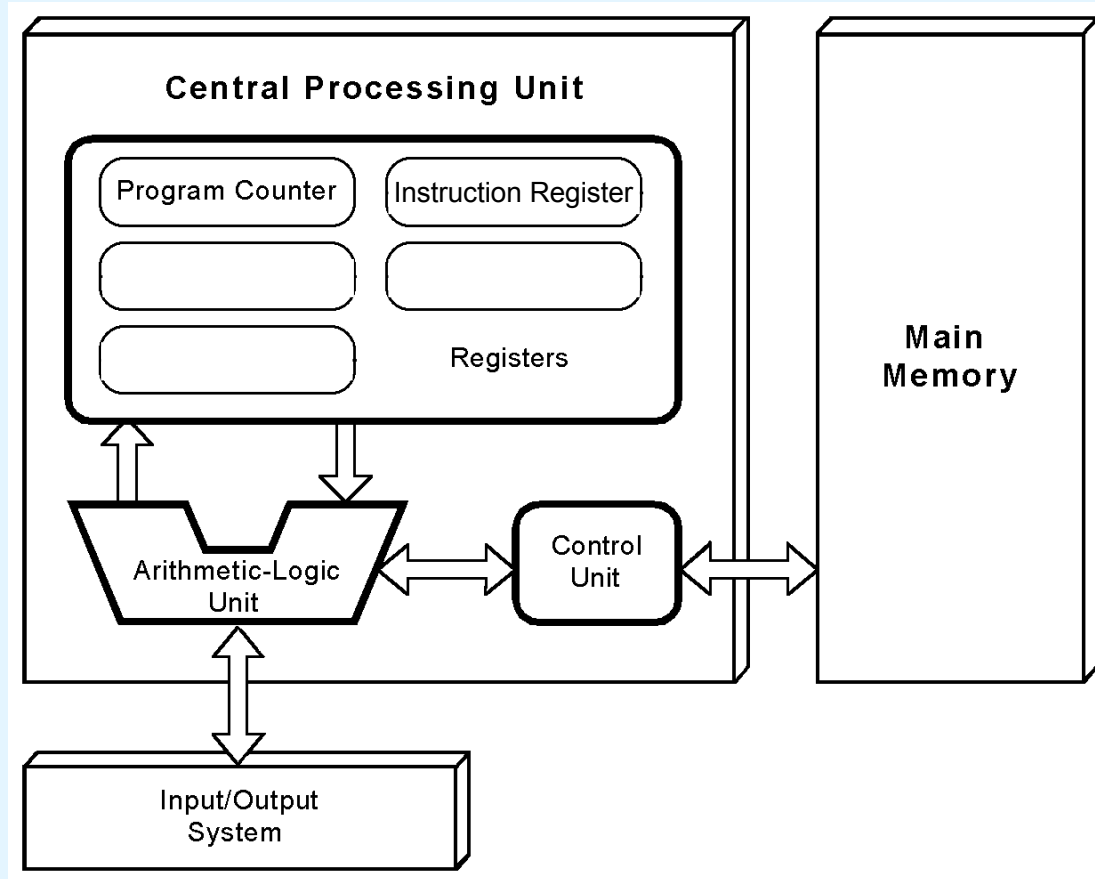
3.0 Introduction



- Unit 1 presented an overview of computer systems.
- Unit 2 discussed how data is stored and manipulated by various computer system components.
- In the Digital Systems subject, we learned about the the fundamental components of digital circuits.
- Having this background, we can now better understand how a CPU based on the von Neumman model works

3.1 Introduction: CPU Basics

- generic scheme of a computer system based on the von Neumann architecture



3.1 Introduction: CPU Basics

- The computer's **CPU** fetches, decodes, and executes program instructions.
- The two main parts of the CPU are the 1) **datapath** and 2) the **control unit**.
 - The *datapath* consists of an arithmetic-logic unit and storage units (registers) that are interconnected by a data bus that is also connected to main memory.
 - Various CPU components perform sequenced operations according to signals provided by its **control unit**.

3.1 Introduction: CPU Basics



- **Registers** are basic storage units that:
 - are inside the CPU
 - have limited storage capacity (8/16/32/64 bits)
 - have very fast access (within a CPU clock-cycle)
 - have multiple functions (roles)
 - store anything (generic / general purpose), store only data, store only addresses, store only state/control information (for instance, to signal ALU error conditions)
 - vary in number (depends on architecture / CPU)
 - the number of generic registers is usually a power of 2
 - are addressed differently from main memory

3.1 Introduction: CPU Basics

- **arithmetic-logic unit (ALU)**
 - carries out logical operations (comparisons, etc.)...
 - ... and arithmetic operations (sums, subtractions, etc.) ...
 - ... as directed by the **control unit**
 - usually has two input registers and one output register
 - operation outcome affects state registers (e.g., overflow)
- **control unit**
 - *fetch* of the instruction pointed by the **Program Counter** register
 - instruction **decode**, including possible operand(s) *fetch*
 - triggers ALU (or other functional units) to **execute** the instruction
 - monitor *status registers*, saves back instruction results
 - attends interrupts (asynchronous notifications of events)

3.1 Introduction: CPU Basics

- **CPU clock**

- Every computer has at least one **system clock** in the motherboard to synchronizes the activities of its components.
- A fixed number of **clock cycles** are required to carry out each data movement or computational operation.
- The **clock frequency** (measured in Hz – cycles per second), sets the the speed with which all operations are carried out.
- **Clock cycle time** is the reciprocal of clock frequency.
 - An 800 MHz clock has a cycle time of $1/(800 \times 10^6) = 1.25 \text{ ns}$.
- The CPU takes the system clock signal and multiplies it to generate an internal signal – the **CPU internal clock**; this signal is used to synchronize all internal activities in the CPU

3.1 Introduction: CPU Basics

- **CPU clock (cont.)**

- Clock speed shouldn't be confused with CPU performance !
- The CPU time required to run a program is given by the general performance equation:

$$\text{CPU Time} = \frac{\text{seconds}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{avg. cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

- Thus, the CPU throughput may be improved by
 - **reducing the number of instructions in a program,**
 - **reducing the number of cycles per instruction,**
 - **reducing the CPU internal clock cycle time.**

We will return later to this important expression.

3.1 Introduction: Basic Memory Organization

- **main memory** (also known as **RAM**)
 - a **vector** (*array*) of **storage cells**, all of the same size
 - the cell size is 8 bits (1 byte) or a multiple
 - each memory cell has **unique address** (integer ≥ 0)
 - with **C** cells, the **address range** is $\{0, 1, \dots, C-1\}$
 - typically, C is a power of 2; therefore, the numbers **0** to **C-1** are written in binary using **$\log_2 C$ bits**
 - if each memory cell stores **B** bits, the memory will have a total storage capacity of **$C \times B$ bits** or **$(C \times B)/8$ bytes**
 - **example:** $C = 4K = 4096 = 2^{12}$: the addresses go from 0 to 4095 and in binary are written with 12 bits; $B = 8$ bits: the total storage capacity of the memory is 4096×8 bits or 4096 bytes = 4Kbytes

3.1 Introduction: MARIE



- We can now bring together many of the ideas that we have discussed to this point using a very simple model computer.
- Our model computer, the **Machine Architecture that is Really Intuitive and Easy** (MARIE), was designed for the singular purpose of illustrating basic computer system concepts.
- While this system is too simple to do anything useful in the real world, a deep understanding of its functions will enable you to comprehend system architectures that are much more complex.

3.2 MARIE: Structure and Organization of the Architecture

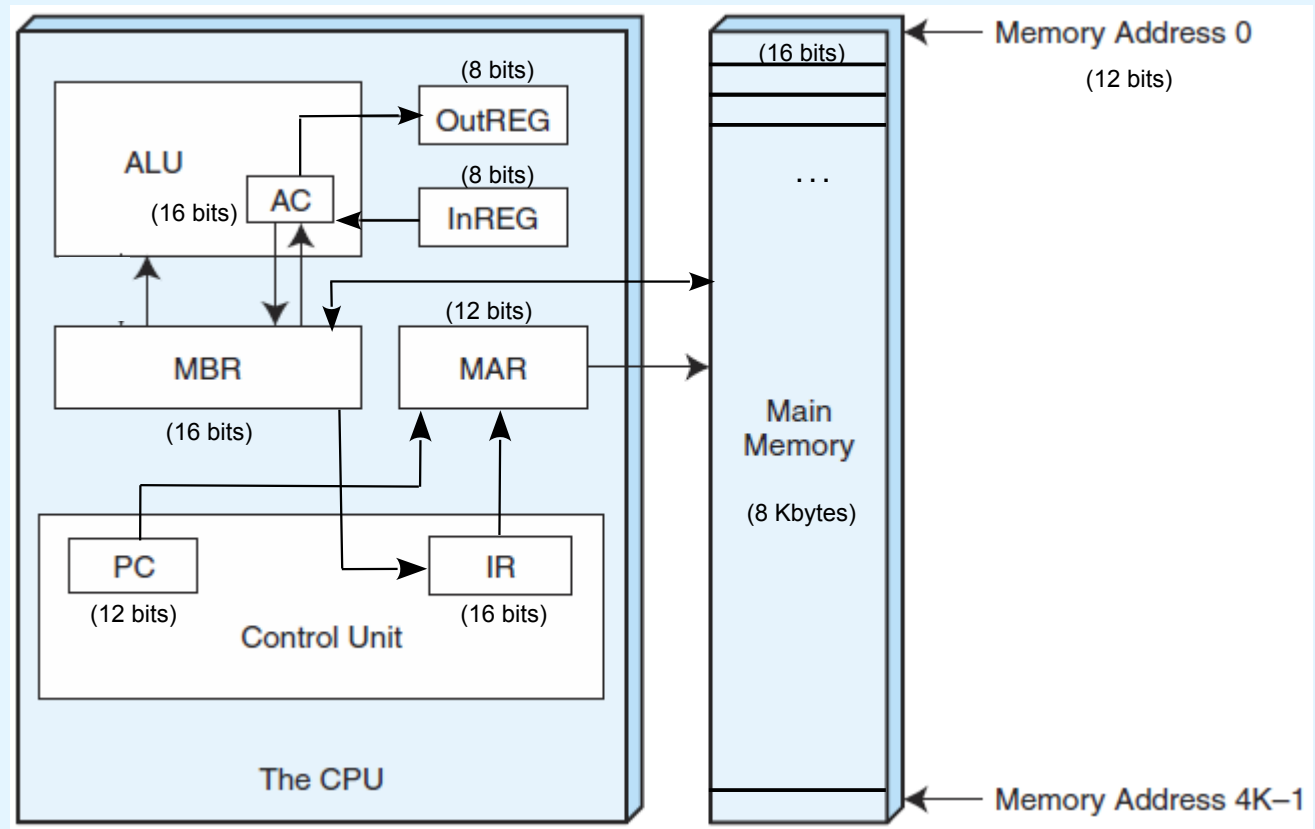
Representation of the Components and Flows

Registers:

AC = Acumulator
InReg = Input Register
OutReg = Output Register

MBR = Memory Buffer Register
MAR = Memory Address Register

PC = Program Counter
IR = Instruction Register



The next slides provide details on the MARIE CPU components and, later, about the flows generated by the fetch-decode-execute cycle.

3.2 MARIE: Structure and Organization of the Architecture

Main Memory:

- MARIE programs stored in a memory of 4K cells (words)
 - addresses from 0 to 4K-1 (addresses are **12** bits wide)
- word size (memory width) = 16 bits
 - data words of 16 bits
 - numbers represented in 2's complement
 - instruction words of 16 bits
 - 4 bits for the opcode, **12** bits for one parameter (the parameter may not be used, may be a memory address, or may be an opcode extension)
- memory capacity: $4K \times 16 \text{ bits} = 8 \text{ Kbytes}$

3.2 MARIE: Structure and Organization of the Architecture

Registers:

- **Accumulator (AC):** a **16-bit** register that holds the result of an operation, or one operand of a two-operand instruction.
- **Memory address register (MAR):** a **12-bit** register that holds the memory address (of an instruction or of the operand of an instruction).
- **Memory buffer register (MBR):** a **16-bit** register that holds the data after its retrieval from, or before its placement in memory.

3.2 MARIE: Structure and Organization of the Architecture

Registers (cont.):

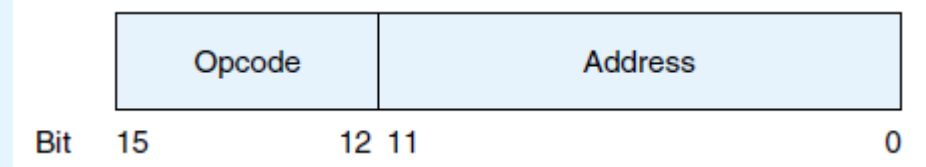
- **Program counter (PC):** a **12-bit** register that holds the address of the next instruction to be executed.
- **Instruction register (IR):** a **16-bit** register that holds an instruction immediately preceding its execution.
- **Input register (InREG):** an **8-bit** register that holds data read from an input device.
- **Output register (OutREG):** an **8-bit** register, that holds data that is ready for the output device.

3.3 MARIE: Instruction Set Architecture

- A computer's ***Instruction Set Architecture (ISA)*** specifies the **format** of its instructions and the **primitive operations** that the machine can perform.
 - The ISA is an interface between a computer's hardware and its software.
 - Some ISAs include 100's of different instructions for processing data and controlling program execution.
 - The MARIE ISA consists of only 13 instructions.
 - 9 fundamental instructions (see next)
 - 6 complementary instructions (introduced later)

3.3 MARIE: Instruction Set Architecture

- This is the format of a MARIE instruction:

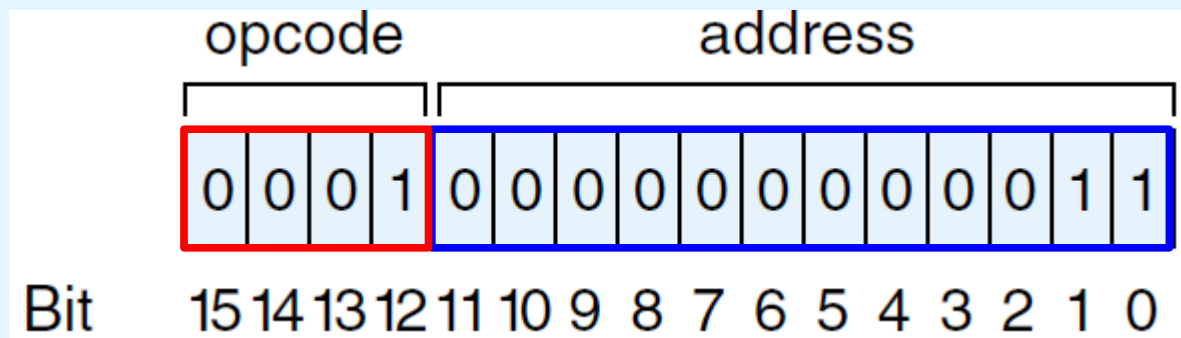


- The fundamental MARIE instructions are:

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

3.3 MARIE: Instruction Set Architecture

- Bit pattern for a **LOAD** **003** instruction in the IR register:



- operation code (*opcode*): $0001(2)=1(16)=1(10)$
- data address: $000000000011(2) = 003(16)=3(10)$
 - in this case, 3 is the final address of the data, once the **direct addressing mode** is being used (later, other addressing modes will be introduced)

checkpoint: exercise MD1

3.4 MARIE: Assembly Process

- Assembly ***mnemonics***, such as **LOAD**, **STORE**, etc., are easy for humans to write and understand, but they are impossible for computers to understand (directly).
- ***Assemblers*** translate **mnemonics** (instructions that are comprehensible to humans) into **machine language** (that is comprehensible to computers)
 - note the distinction between an **assembler** and a **compiler**:
 - In assembly language, there is a one-to-one correspondence between a mnemonic instruction and its machine code.
 - With compilers, this is not usually the case.

3.4 MARIE: Assembly Process

- Assemblers create an *object program file* from mnemonic *source code* in two passes.
- **First pass:** the assembler translates as much of the program as it can, and builds a ***symbol table*** with memory references for all symbols in the program.
- **Second pass:** the instructions are completed using the values from the symbol table.

3.4 MARIE: Assembly Process

- Consider the example program to the right:
 - Note that we have included two directives **HEX** and **DEC** that specify the radix of the constants.
- The first pass produces the symbol table and the partial machine code:

// example 3.0

Address	Instruction
100	Load X
101	Add Y
102	Store Z
103	Halt
104 X,	DEC 35
105 Y,	DEC -23
106 Z,	HEX 0000

X	104
Y	105
Z	106

1	X
3	Y
2	Z
7	0000

3.4 MARIE: Assembly Process

- After the second pass, the assembly is complete :

1	1	0	4
3	1	0	5
2	1	0	6
7	0	0	0
0	0	2	3
F	F	E	9
0	0	0	0

X	104
Y	105
Z	106

// example 3.0

Address	Instruction
100	Load X
101	Add Y
102	Store Z
103	Halt
104 X,	DEC 35
105 Y,	DEC -23
106 Z,	HEX 0000

checkpoint: exercises
MAT1.a, MAT2.a,
MAT3.a, MAT4.a,
MAT5.a, MAT6.1,

3.5 MARIE: Register Transfer Language

- Each instructions actually consists of a sequence of smaller instructions called ***micro-operations***.
- The exact sequence of *micro-operations* that are carried out by an instruction can be specified using ***register transfer language (RTL)***.
- In the MARIE RTL, the following notation is used:
 - **$M[X]$** : data value stored in memory location X
 - $\text{destiny} \leftarrow \text{source}$: transfer from source (*) to destiny (*); (*)=CPU register or memory address

3.5 MARIE: Register Transfer Language

Opcode	Instruction	RTN
0001	Load X	$MAR \leftarrow X$ (*) $MBR \leftarrow M[MAR], AC \leftarrow MBR$
0010	Store X	$MAR \leftarrow X$ (*) $MBR \leftarrow AC, M[MAR] \leftarrow MBR$
0011	Add X	$MAR \leftarrow X$ (*) $MBR \leftarrow M[MAR]$ $AC \leftarrow AC + MBR$
0100	Subt X	$MAR \leftarrow X$ (*) $MBR \leftarrow M[MAR]$ $AC \leftarrow AC - MBR$
0101	Input	$AC \leftarrow InREG$
0110	Output	$OutREG \leftarrow AC$
0111	Halt	
1000	Skipcond	If IR[11-10] = 00 then If AC < 0 then PC $\leftarrow$ PC + 1 Else If IR[11-10] = 01 then If AC = 0 then PC $\leftarrow$ PC + 1 Else If IR[11-10] = 10 then If AC > 0 then PC $\leftarrow$ PC + 1
1001	Jump X	PC $\leftarrow$ IR[11-0]

(*) during decode

// Skipcond 000

// Skipcond 400

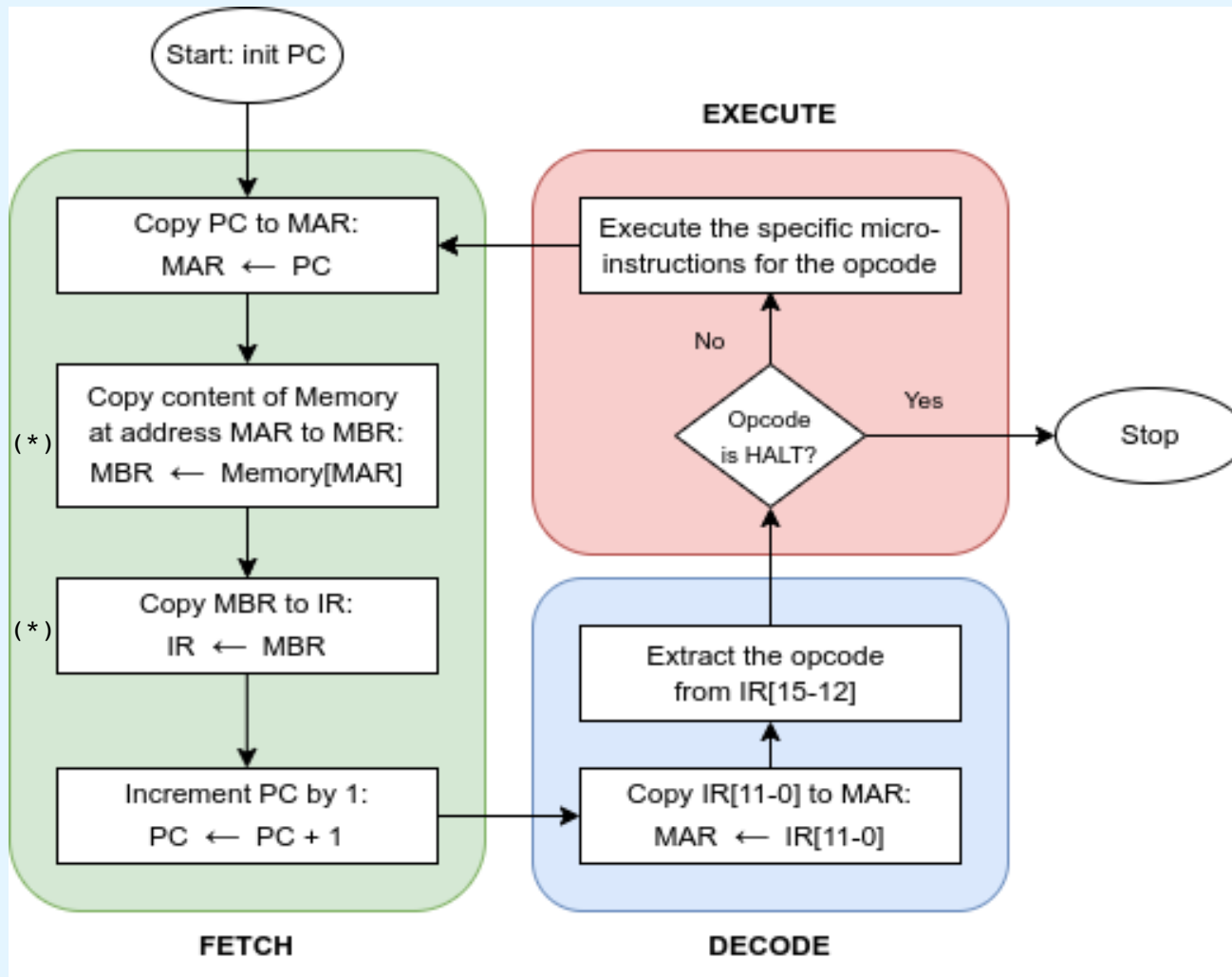
// Skipcond 800

3.6 MARIE: fetch-decode-execute cycle

- The ***fetch-decode-execute*** cycle in MARIE is the series of steps it carries out when it runs a program:
 - step 1: ***fetch*** an instruction from memory into the IR
 - step 2: ***decode*** the instruction to determine its nature and check if all its parameters are present
 - if a memory value (operand) is necessary, read it
 - step 3: ***execute*** the instruction
 - this may imply writing back into memory (e.g., Store X, which saves the content of AC in the memory cell X)

The next slide shows a flowchart of this process.

3.6 MARIE: fetch-decode-execute cycle



3.7 MARIE: Trace Diagrams

- **Trace Diagrams:** show what happens inside the MARIE CPU during the execution of a program
- The next slides provide trace diagrams for the program previously assembled (example 3.0)

Hex Address	Instruction	Binary Contents of Memory Address	Hex Contents of Memory
100	Load 104	0001000100000100	1104
101	Add 105	0011000100000101	3105
102	Store 106	0010000100000110	2106
103	Halt	0111000000000000	7000
104	0023	0000000000100011	0023
105	FFE9	1111111111101001	FFE9
106	0000	0000000000000000	0000

3.7 MARIE: Trace Diagrams

- LOAD 104:**

Hex Address	Instruction	Hex Contents of Memory
100	Load 104	1104
101	Add 105	3105
102	Store 106	2106
103	Halt	7000
104	0023	0023
105	FFE9	FFE9
106	0000	0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		100	----	---	----	----
Fetch	MAR $\leftarrow$ PC	100	----	100	----	----
	MBR $\leftarrow$ M[MAR](*)	100	----	100	1104	----
	IR $\leftarrow$ MBR (*)	100	1104	100	1104	----
	PC $\leftarrow$ PC + 1	101	1104	100	1104	----
Decode	MAR $\leftarrow$ IR[11-0]	101	1104	104	1104	----
	Decode IR[15-12]	101	1104	104	1104	----
Execute	MBR $\leftarrow$ M[MAR]	101	1104	104	0023	----
	AC $\leftarrow$ MBR	101	1104	104	0023	0023

(*) in the text book, the instruction does not pass through the MBR and is copied directly to the IR

3.7 MARIE: Trace Diagrams

- **ADD 105:**

Hex Address	Instruction	Hex Contents of Memory
100	Load 104	1104
101	Add 105	3105
102	Store 106	2106
103	Halt	7000
104	0023	0023
105	FFE9	FFE9
106	0000	0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		101	1104	104	0023	0023
Fetch	MAR $\leftarrow$ PC	101	1104	101	0023	0023
	MBR $\leftarrow$ M[MAR](*)	101	1104	101	3105	0023
	IR $\leftarrow$ MBR (*)	101	3105	101	3105	0023
	PC $\leftarrow$ PC + 1	102	3105	101	3015	0023
Decode	MAR $\leftarrow$ IR[11-0]	102	3105	105	3105	0023
	Decode IR[15-12]	102	3105	105	3105	0023
Execute	MBR $\leftarrow$ M[MAR]	102	3105	105	FFE9	0023
	AC $\leftarrow$ AC + MBR	102	3105	105	FFE9	000C

(*) in the text book, the instruction does not pass through the MBR and is copied directly to the IR

3.7 MARIE: Trace Diagrams

- STORE 106:**

Hex Address	Instruction	Hex Contents of Memory
100	Load 104	1104
101	Add 105	3105
102	Store 106	2106
103	Halt	7000
104	0023	0023
105	FFE9	FFE9
106	0000	0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		102	3105	105	FFE9	000C
Fetch	MAR $\leftarrow$ PC	102	3105	102	FFE9	000C
	MBR $\leftarrow$ M[MAR] (*)	102	3105	102	2106	000C
	IR $\leftarrow$ MBR (*)	102	2106	102	2106	000C
	PC $\leftarrow$ PC + 1	103	2106	102	2106	000C
Decode	MAR $\leftarrow$ IR[11-0]	103	2106	106	2106	000C
	Decode IR[15-12]	103	2106	106	2106	000C
Execute	MBR $\leftarrow$ AC	103	2106	106	000C	000C
	M[MAR] $\leftarrow$ MBR	103	2106	106	000C	000C

(*) in the text book, the instruction does not pass through the MBR and is copied directly to the IR

3.7 MARIE: Trace Diagrams

- HALT:**

Hex Address	Instruction	Hex Contents of Memory
100	Load 104	1104
101	Add 105	3105
102	Store 106	2106
103	Halt	7000
104	0023	0023
105	FFE9	FFE9
106	0000	0000

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		103	2106	106	000C	000C
Fetch	MAR $\leftarrow$ PC	103	2106	103	000C	000C
	MBR $\leftarrow$ M[MAR] _(*)	103	2106	103	7000	000C
	IR $\leftarrow$ MBR _(*)	103	7000	103	7000	000C
	PC $\leftarrow$ PC + 1	104	7000	103	7000	000C
Decode	MAR $\leftarrow$ IR[11-0]	104	7000	000	7000	000C
	Decode IR[15-12]	104	7000	000	7000	000C
Execute	Stop	104	7000	000	7000	000C

(*) in the text book, the instruction does not pass through the MBR and is copied directly to the IR

3.7 MARIE: Trace Diagrams

Hex Address	Instruction	Hex Contents of Memory
100	Load 104	1104
101	Add 105	3105
102	Store 106	2106
103	Halt	7000
104	0023	0023
105	FFE9	FFE9
106	0000	0000

Tracing (alternative representation):

	Load 104	Add 105	Store 106	Halt
MAR:	-,100,104	104,101,105	105,102,106	106,103,000
MBR:	-,1104,0023	0023,3105,FFE9	FFE9,2106,000C	000C,7000
PC:	100,101	101,102	102,103	103,104
IR:	-,1104	1103,3105	3105,2106	2106,7000
AC:	-,0023	0023,000C	000C=000C	000C=000C
red : initial valued		blue : final value		

Tracing (compact alternative representation):

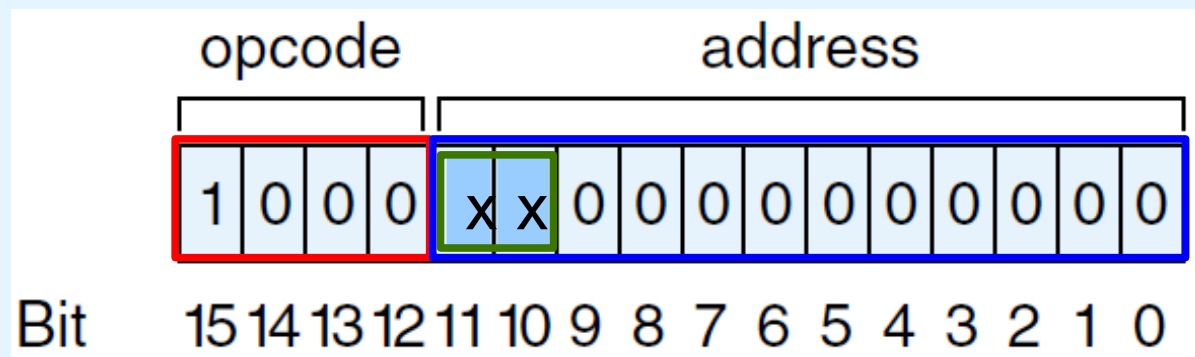
	Load 104	Add 105	Store 106	Halt
MAR:	100,104	101,105	102,106	103,000
MBR:	1104,0023	3105,FFE9	2106,000C	7000
PC:	100,101	102	103	104
IR:	1104	3105	2106	7000
AC:	0023	000C	000C	000C
blue : final value				

checkpoint: test the simulator (<https://marie.v2.pt>) with example 3.0; exercises MAT1.b, MAT2.b, MAT3.b, MAT4.b, MAT5.b, MAT6.2

3.8 MARIE

- Conditional Branching (If-Then-Else)

- in MARIE, conditional branching of the type If-Then-Else builds on the use of the instruction SKIPCOND
- bit pattern of **SKIPCOND** in the IR register:



- operation code (*opcode*): $1000(2)=8(16)=8(10)$
- if the bits in positions 11 and 10 take the values “00”, “01”, “00”, then the next instruction will not be executed (PC++) provided the value of AC is $<$, $=$ or $>$ 0

3.8 MARIE

- Conditional Branching (If-Then-Else)

```
// example 3.1
int x=12, y=20;
if (x==y) x = 2 * x;
else y = y - x;
printf("%d", y);
```

checkpoint:
exercises MC1 to MC5

If,	Load	X	/Load the first value
	Subt	Y	/Subtract value of Y and store result in AC
	Skipcond	400	/If AC = 0, skip the next instruction
	Jump	Else	/Jump to the Else part if AC is not equal to 0
Then,	Load	X	/Reload X so it can be doubled
	Add	X	/Double X
	Store	X	/Store the new value
	Jump	Endif	/Skip over Else part to end of If
Else,	Load	Y	/Start the Else part by loading Y
	Subt	X	/Subtract X from Y
	Store	Y	/Store Y - X in Y
Endif,	Load	Y	/Reload Y so it can be printed
	Output		/Print Y
	Halt		/Terminate program
X,	Dec	12	/Assume this initial value for X
Y,	Dec	20	/Assume this initial value for Y

3.9 MARIE: Complementary Instructions

- So far, all of the MARIE instructions that were discussed use a ***direct*** addressing mode:
 - the instruction includes the address of the operand
- Other instructions employ ***indirect*** addressing:
 - the instruction includes the address of a memory cell (*), which stores the address of the intended operand

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

(*) i.e., this memory cell is in fact a pointer !

3.9 MARIE: Complementary Instructions

- RTL Description

JnS

$MAR \leftarrow X (*)$

$MBR \leftarrow PC$

$M[MAR] \leftarrow MBR$

$MBR \leftarrow X$

$AC \leftarrow 1$

$AC \leftarrow AC + MBR$

$PC \leftarrow AC$

Clear

$AC \leftarrow 0$

Addl X

$MAR \leftarrow X (*)$

$MBR \leftarrow M[MAR]$

$MAR \leftarrow MBR$

$MBR \leftarrow M[MAR]$

$AC \leftarrow AC + MBR$

Jumpl X

$MAR \leftarrow X (*)$

$MBR \leftarrow M[MAR]$

$PC \leftarrow MBR$

Loadl X

$MAR \leftarrow X (*)$

$MBR \leftarrow M[MAR]$

$MAR \leftarrow MBR$

$MBR \leftarrow M[MAR]$

$AC \leftarrow MBR$

Storel X

$MAR \leftarrow X (*)$

$MBR \leftarrow M[MAR]$

$MAR \leftarrow MBR$

$MBR \leftarrow AC$

$M[MAR] \leftarrow MBR$

(*) done during decode

3.9 MARIE: Complementary Instructions

- Subroutines: JnS X

- **JNS X** (*jump-and-store*) : used together with **JUMPI X** , to support the execution of **subroutines**
 - save the address **X** in PC and jump to **X+1**, where the subroutine starts; the subroutine returns via **JUMPI X**
 - RTL description:

```
MBR ← PC
MAR ← X
M[MAR] ← MBR
MBR ← X
AC ← 1
AC ← AC + MBR
PC ← AC
```


3.9 MARIE: Complementary Instructions

- Subroutines: JumpI X

- **JUMPI X** (indirect jump) :
 - “the value to assign to PC is the one stored in X”
 - X contains the address of the next instruction to execute
 - JUMPI 400 : if the memory cell 400 contains the value 240, the next instruction to execute is the one stored in memory cell 240
 - RTL description:

MAR $\leftarrow$ X

MBR $\leftarrow$ M[MAR]

PC $\leftarrow$ MBR

3.9 MARIE: Complementary Instructions

- Subroutines (example)

ORG 100

```
Load X      / load the 1st number (X) to be doubled
Store Num   / use Num as a parameter to pass X to Twice
JnS Twice   / store return address and jump to Twice
Load Res    / load the double of X returned by Twice
Store X     / update X with its double
Output      / show the new value of X
```

```
Load Y      / load the 2nd number (Y) to be doubled
Store Num   / use Num as a parameter to pass Y to Twice
JnS Twice   / store return address and jump to Twice
Load Res    / load the double of Y returned by Twice
Store Y     / update Y with its double
Output      / show the new value of Y
```

```
Halt        / end program
```

```
Twice, Hex 0      / place to store return address
Load Num    / AC = Num
Add Num     / AC = AC + Num = Num + Num = 2 * Num
Store Res   / Res = 2 * Num
JumpI Twice / return to calling code
```

```
// example 3.3
int twice (int num) { int res;
res = num + num; return(res); }
main() {
    int x = 20, y = 48;
    x = twice(x); printf("%d", x);
    y = twice(y); printf("%d", y);
}
```

checkpoint:
exercises (MC7),
MC8, MC11.a(b),
MC11.c, MC9+10,
MC11.d, MC11.e+f

X,	Dec	20
Y,	Dec	48
Num,	Dec	0
Res,	Dec	0

3.9 MARIE: Complementary Instructions

- Load X

- **LOADI X** (indirect load):
 - “the value to be read to AC is the one pointed by X”
 - X contains the address of the memory cell from which to read the value to assign to the register AC
 - **LOADI 400** : if the memory cell 400 contains the value 240, the value to assign to the register AC will be read from the memory cell 240
 - RTL description:
$$\begin{aligned} \text{MAR} &\leftarrow X \\ \text{MBR} &\leftarrow \text{M}[\text{MAR}] \\ \text{MAR} &\leftarrow \text{MBR} \\ \text{MBR} &\leftarrow \text{M}[\text{MAR}] \\ \text{AC} &\leftarrow \text{MBR} \end{aligned}$$

3.9 MARIE: Complementary Instructions

- Addl X

- **ADDI X** (indirect addition):
 - “the value to be added to AC is the one pointed by X”
 - X contains the address of the memory cell from which to read the value to add to the register AC
 - ADDI 400 : if the memory cell 400 contains the value 240, the value to add to the register AC will read from the memory cell 240
 - RTL description (similar to **LOADI X**):

MAR $\leftarrow$ **X**

MBR $\leftarrow$ **M**[**MAR**]

MAR $\leftarrow$ **MBR**

MBR $\leftarrow$ **M**[**MAR**]

AC $\leftarrow$ **AC** + **MBR**

3.9 MARIE: Complementary Instructions

- LoadI, AddI (example)

```
// example 3.2
char a[13]="Hello World!";
int i=0;
while (a[i]!='\0') {
    printf("%c", a[i]);
    i = i + 1;
}
```

```
ORG 100

Loop,   Load   ArrayBaseAddress
        Add     ArrayCellIndex
        Store   ArrayCellAddress / new address of a[i]

        LoadI   ArrayCellAddress
        Skipcond 400           / check if a[i] == '\0'
        Jump    BodyLoop
        Jump    EndLoop

BodyLoop, Output / output a[i]

        Load     ArrayCellIndex
        Add       One
        Store     ArrayCellIndex / i++

        Jump     Loop

EndLoop, Halt
```

```
/ alternative
Clear
AddI ArrayCellAddress
```

Array,	Dec	072	/ H
	Dec	101	/ e
	Dec	108	/ l
	Dec	108	/ l
	Dec	111	/ o
	Dec	032	/ [space]
	Dec	119	/ w
	Dec	111	/ o
	Dec	114	/ r
	Dec	108	/ l
	Dec	100	/ d
	Dec	033	/ !
	Dec	000	/ [null]
ArrayBaseAddress, Hex 10D / address of a[0]			
/ArrayBaseAddress, Hex 10E / alternative			
ArrayCellAddress, Hex 000 / address of a[i]			
ArrayCellIndex,	Dec	0	/ index i
One,	Dec	1	/ constant 1

checkpoint:
exercises MC6,
MC12.a(b)c/d

3.9 MARIE: Complementary Instructions

- LoadI, AddI (example)

```
// example 3.3
int a[5]={10,15,20,25,30};
int sum=0, i=0;
while (i<5) {
    sum = sum + a[i];
    i = i + 1;
}
printf("%d", sum);
```

```

ORG 100

Loop,   Load      ArrayCellIndex
        Subt       Five
        Skipcond   000           / check if i < 5
        Jump       EndLoop

        Load       ArrayBaseAddress
        Add         ArrayCellIndex
        Store       ArrayCellAddress / new address of a[i]

        Load       Sum
        AddI        ArrayCellAddress
        Store       Sum           / sum = sum + a[i]

        Load       ArrayCellIndex
        Add         One
        Store       ArrayCellIndex / i++

        Jump       Loop

EndLoop, Load   Sum
        Output
        Halt

```

```

/ alternative
/ sum = a[i] + sum
LoadI ArrayCellAddress
Add    Sum
Store  Sum

```

```

Dec 10 / a[0]
Dec 15 / a[1]
Dec 20 / a[2]
Dec 25 / a[3]
Dec 30 / a[4]
ArrayBaseAddress, Hex 111 / address of a[0]
ArrayCellAddress, Hex 000 / address of a[i]
ArrayCellIndex,   Dec 0   / index i
Sum,              Dec 0   / sum
One,              Dec 1   / constant 1
Five,            Dec 5   / constant 5

```

3.9 MARIE: Complementary Instructions

- StoreI X

- **STOREI X** (indirect store):
 - “the value of AC is stored in the memory cell pointed by X”
 - X contains the address of the memory cell where to store the current value of the register AC
 - STOREI 400 : if the memory cell 400 contains the value 240, the current value of the register AC will be stored in memory cell 240
 - RTL description:

$$\begin{aligned}\text{MAR} &\leftarrow X \\ \text{MBR} &\leftarrow \text{M}[\text{MAR}] \\ \text{MAR} &\leftarrow \text{MBR} \\ \text{MBR} &\leftarrow \text{AC} \\ \text{M}[\text{MAR}] &\leftarrow \text{MBR}\end{aligned}$$

3.9 MARIE: Complementary Instructions

- Storel (example)

```
// example 3.4
int a[5], i;
for (i=0; i<5; i++) {
    scanf("%d", &a[i]);
}
```

checkpoint:
exercises
MC13.a/b,
MATC1

```

ORG 100

Loop,   Load      ArrayCellIndex
        Subt       Five
        Skipcond   000           / check if i < 5
        Jump      EndLoop

        Load      ArrayBaseAddress
        Add       ArrayCellIndex
        Store     ArrayCellAddress / new address of a[i]

        Input
        StoreI    ArrayCellAddress / read a[i] from user

        Load      ArrayCellIndex
        Add       One
        Store     ArrayCellIndex  / i++

        Jump      Loop

EndLoop, Halt

```

```

Dec 0 / a[0]
Dec 0 / a[1]
Dec 0 / a[2]
Dec 0 / a[3]
Dec 0 / a[4]
ArrayBaseAddress, Hex 10E / address of a[0]
ArrayCellAddress, Hex 000 / address of a[i]
ArrayCellIndex,   Dec 0   / index i
One,              Dec 1   / constant 1
Five,             Dec 5   / constant 5

```


3.10 Instruction Decoding

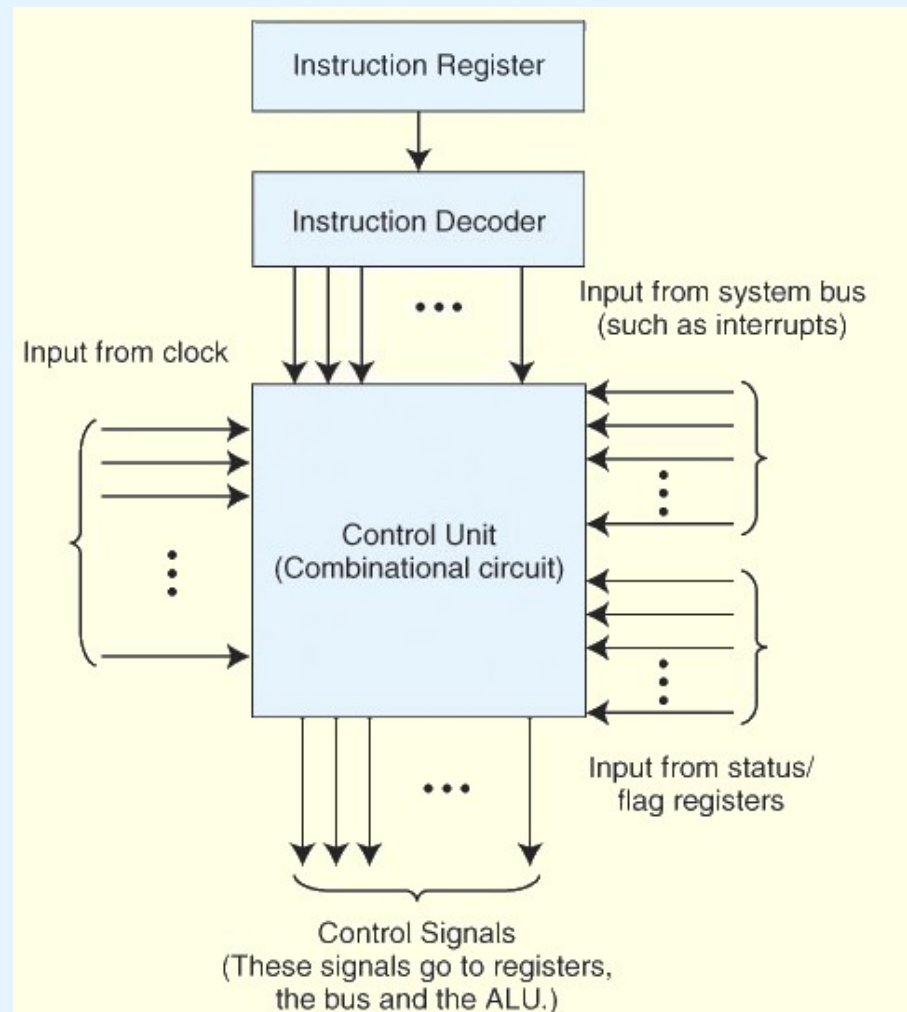


- A computer's control unit keeps things synchronized, making sure that bits flow to the correct components as needed.
- Two general ways to implement a control unit (CU):
 - ***Hardwired control***: the CU is a combinatorial circuit; for a certain combination of input bits, it asserts a specific set of output lines; these lines control the behavior of other digital logic components.
 - ***Microprogrammed control***: the CU runs a small program (from a read-only memory) that converts each machine instruction in a set of micro-instructions; these in turn are translated to electric control signals

3.10 Instruction Decoding

Hardwired control:

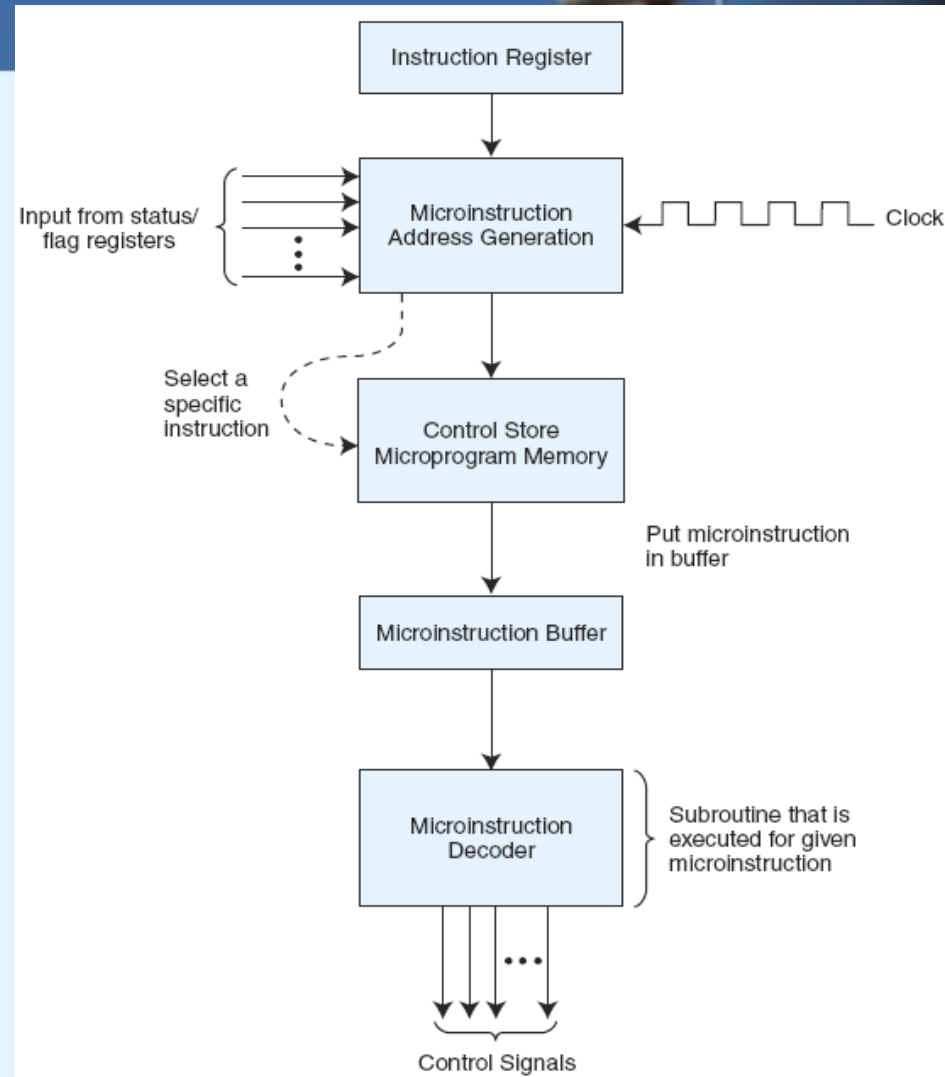
- in order to decode MARIE's *opcodes*, it would be necessary a 4-to-16 decoder
- the diagram to the right shows a generic configuration for a control unit that implements hardwired control
- (+) : very good performance
- (-) : complex, inflexible



3.10 Instruction Decoding

Microprogrammed control:

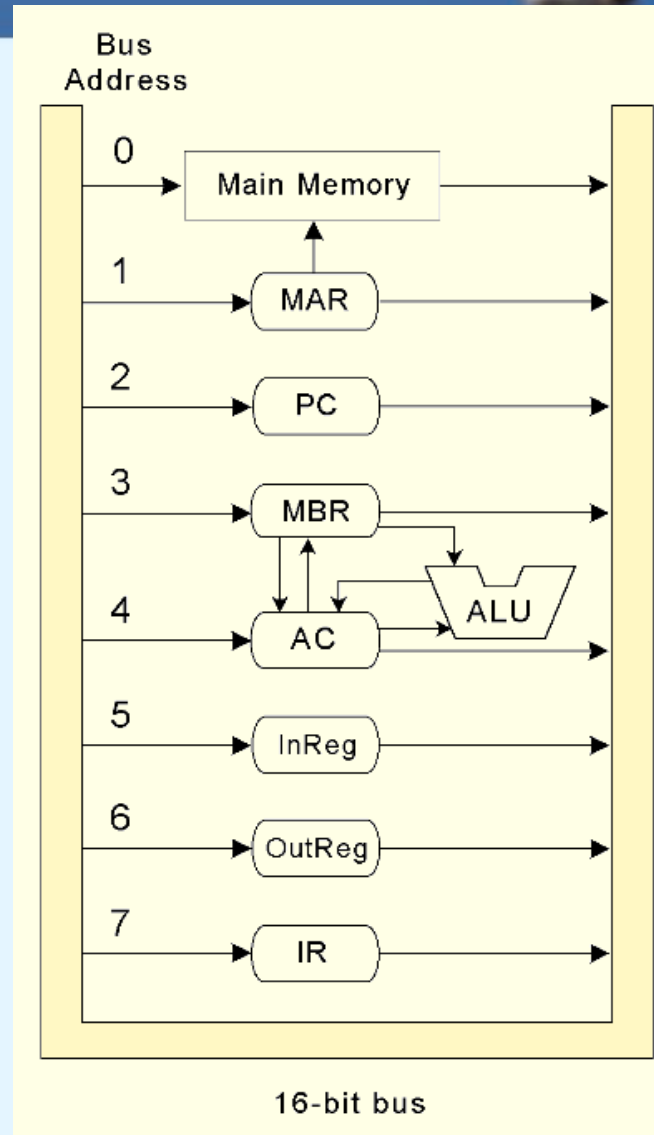
- the interpreter (microprogram) is stored as *firmware* in a memory (ROM, PROM or EPROM), also known as the *control store*
- each machine instruction unfolds into several micro-instructions; there is a “subroutine” for each micro-instruction, that will assert the proper digital components
- (-) : < performance (in software)
- (+) : simple, flexible (extending the *instruction-set* is achieved by coding new sub-routines)



3.10.1 Instruction Decoding in MARIE

- Registers and Main Memory have a unique address along MARIE's datapath →
 - the addresses take the form of signals issued by the control unit

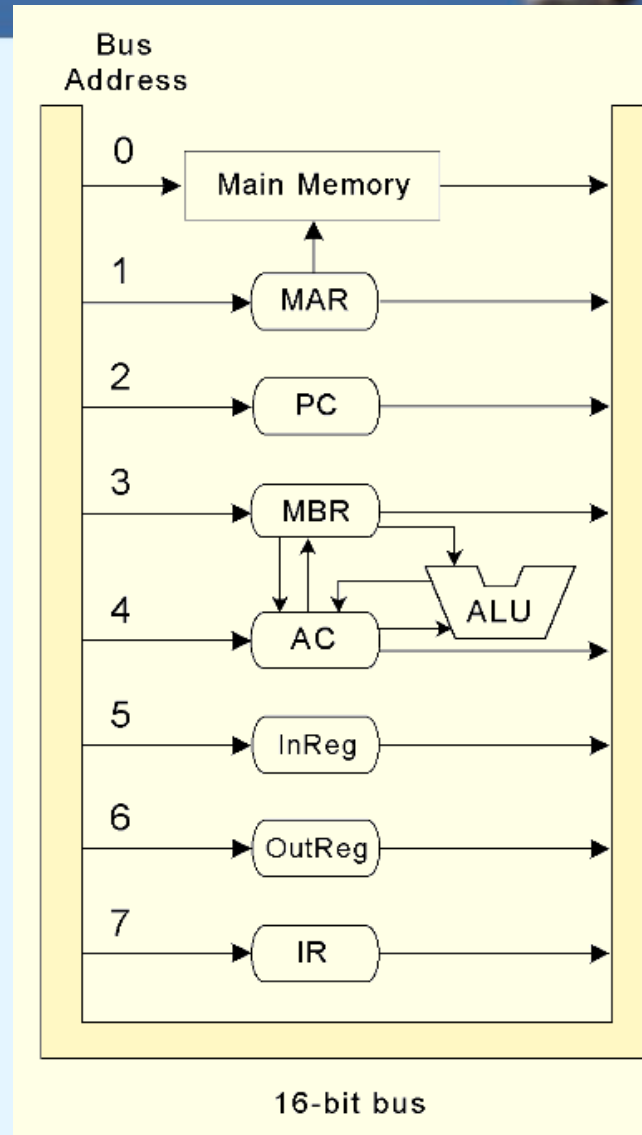
How many signal lines does MARIE's control unit need?



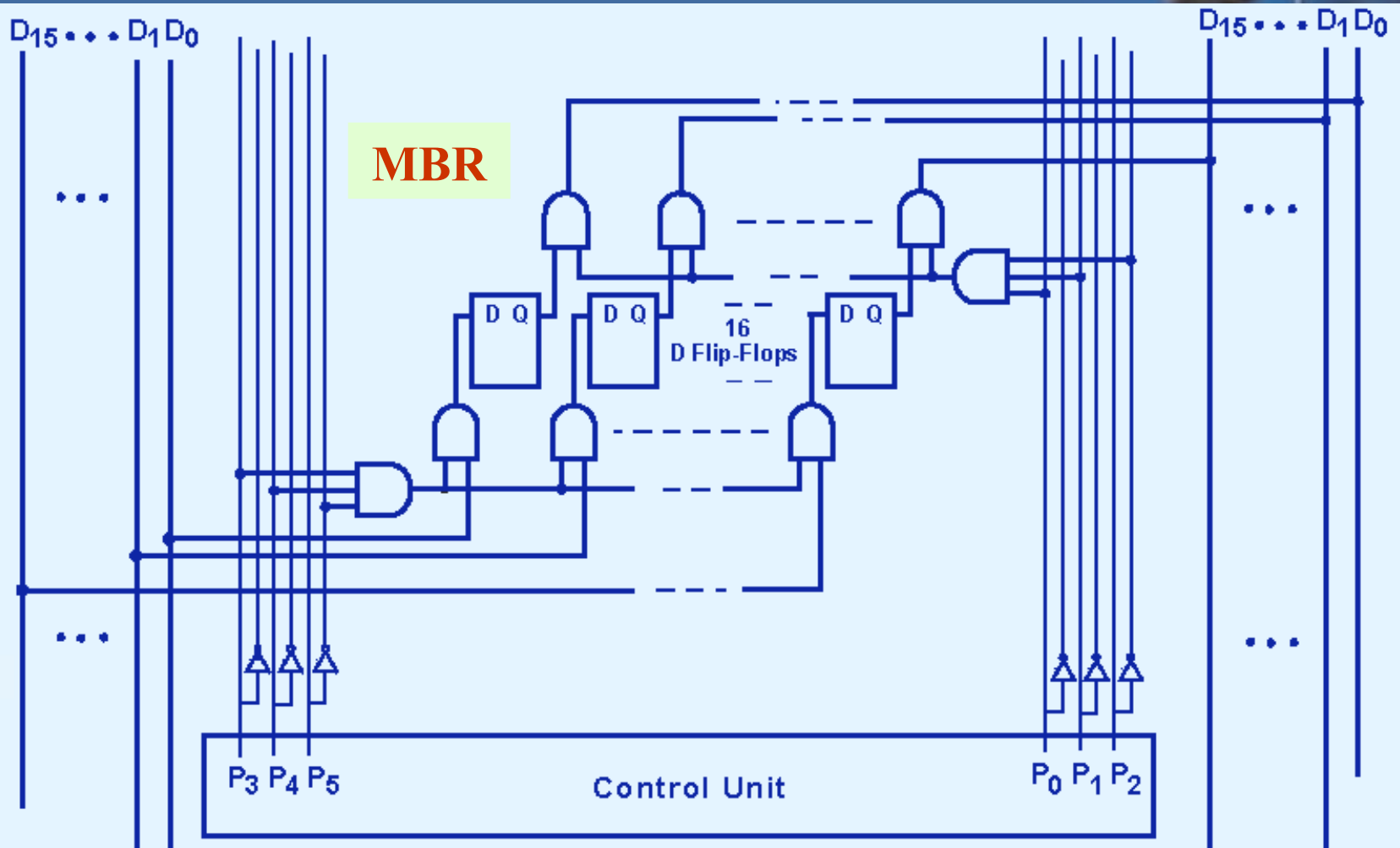
3.10.1 Instruction Decoding in MARIE

- Two sets of three signals:
 - One set, $\{P_0, P_1, P_2\}$, controls reading from memory or a register;
 - The other set, $\{P_3, P_4, P_5\}$, controls writing to memory or a register.

The next slide shows a close up view of MARIE's MBR.



3.10.1 Instruction Decoding in MARIE



The MBR is enabled for reading (writing) when P₀ and P₁ (P₃ and P₄) are high (note that MBR's bus address is 3=011)

3.10.1 Instruction Decoding in MARIE

- Inspection of MARIE's RTL reveals that the ALU has only three operations: add, subtract, clear.
 - We will also define a fourth “do nothing” state.
- The entire set of MARIE's control signals would be:
 - Register controls: P_0 through P_5 .
 - ALU controls: A_0 through A_3
 - Timing: T_0 through T_7 and counter reset C_r

3.10.1 Instruction Decoding in MARIE

- Consider MARIE's "Add X" instruction. It's RTL is:
 MAR $\leftarrow$ **X**
 MBR $\leftarrow$ **M**[**MAR**]
 AC $\leftarrow$ **AC** + **MBR**
- Let's focus on the first step (**MAR** $\leftarrow$ **X**):
 - After the fetch, the address, **X**, is in the rightmost 12 bits of the IR, which has a datapath address of 7.
 - **X** is copied to the MAR, which has a datapath address of 1.
 - Thus we need to raise signals **P**₂, **P**₁, and **P**₀ (111=7) to read from the IR, and signal **P**₃ (001=1) to write to the MAR.

3.10.1 Instruction Decoding in MARIE

- The complete signal sequence for Add would be:

$P_0 P_1 P_2 P_3 T_0: \text{MAR} \leftarrow X$

$P_0 P_2 T_1: \text{MBR} \leftarrow M[\text{MAR}]$

$A_0 P_0 P_1 P_2 P_5 T_2: \text{AC} \leftarrow \text{AC} + \text{MBR}$

$C_r T_3: [\text{Reset counter}]$

- These signals are ANDed with combinational logic to bring about the desired machine behavior.

The next slide shows the timing diagram for Add.

3.10.1 Instruction Decoding in MARIE

- Notice the concurrent signal states during each machine cycle (cycles C_0 through C_3)

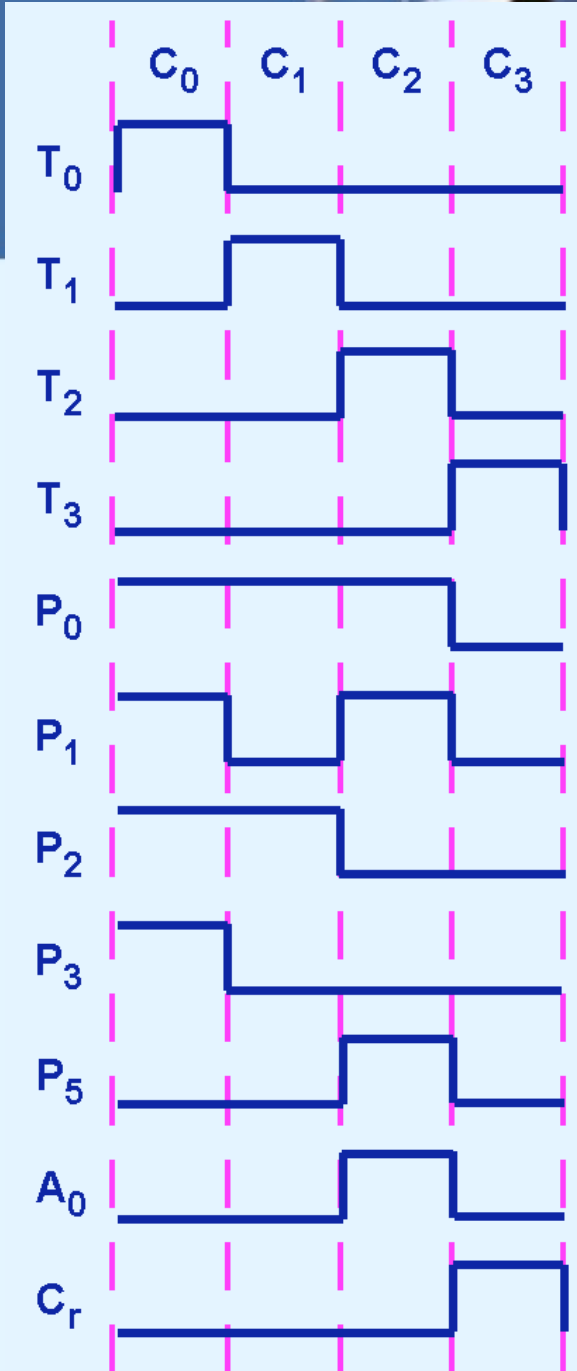
$P_0 P_1 P_2 P_3 T_0: MAR \leftarrow X$

$P_0 P_2 T_1: MBR \leftarrow M[MAR]$

$A_0 P_0 P_1 P_2 P_5 T_2: AC \leftarrow AC + MBR$

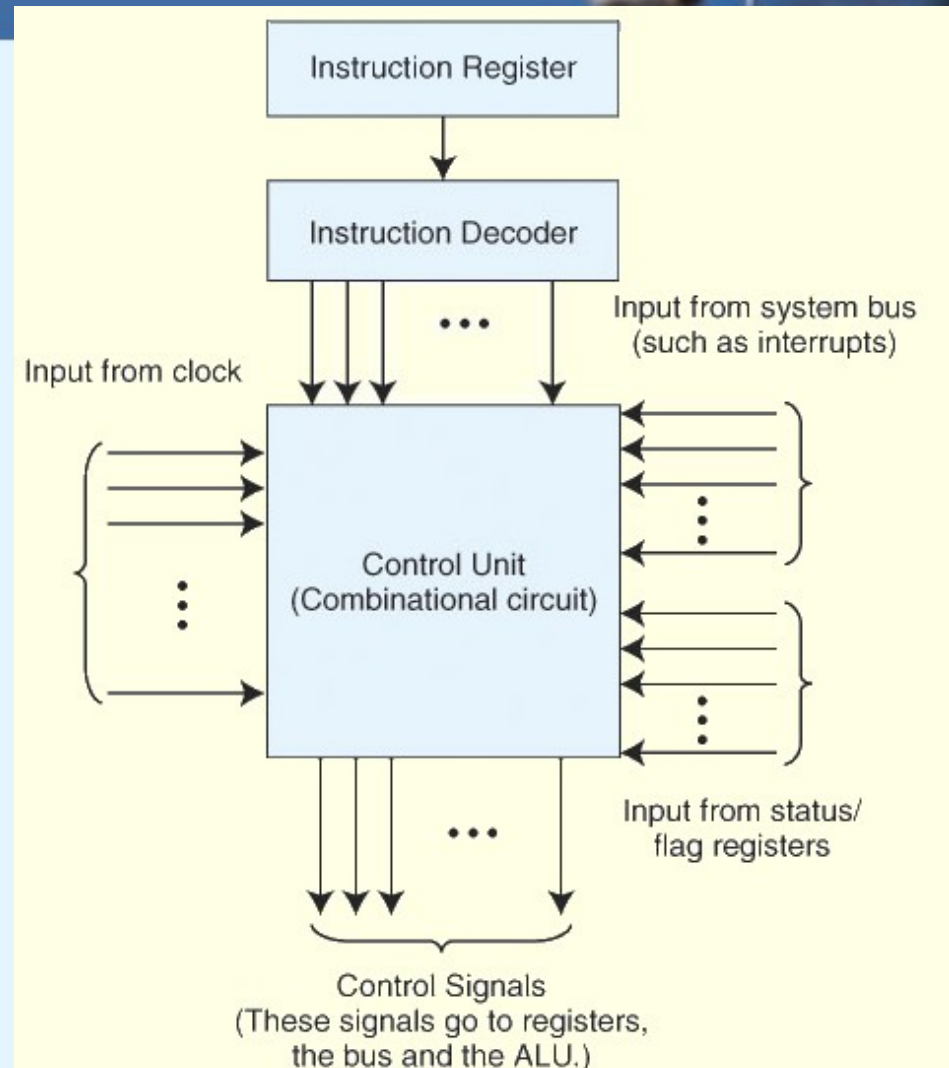
$C_r T_3: [\text{Reset counter}]$

In MARIE, the Add instruction needs 4 machine-cycles

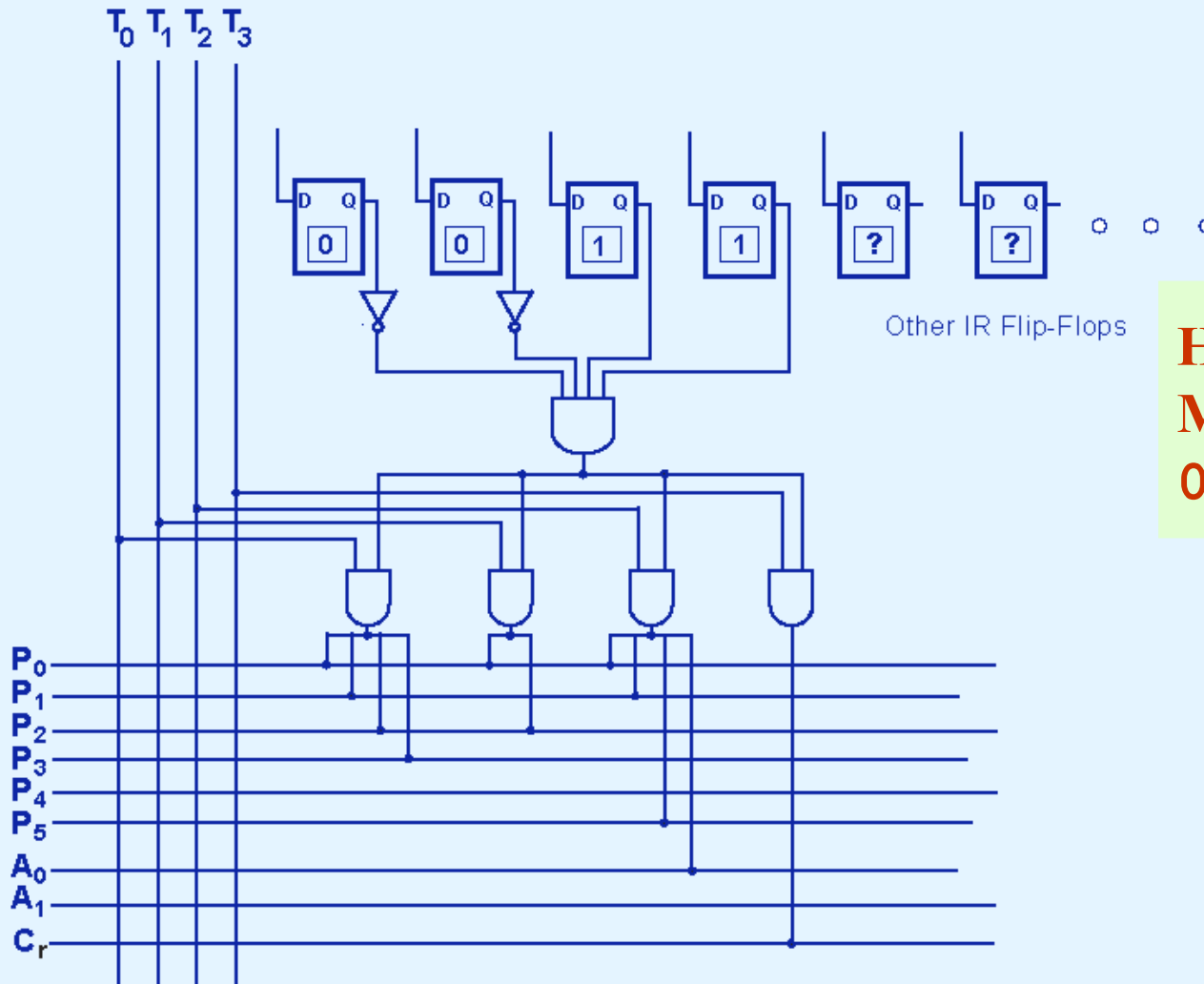


3.10.1 Instruction Decoding in MARIE

- The signal pattern just described is the same whether our machine used hardwired control or microprogrammed control.
- In **hardwired control**, the bit pattern of an instruction feeds directly into the combinatorial logic within the control unit.



3.10.1 Instruction Decoding in MARIE

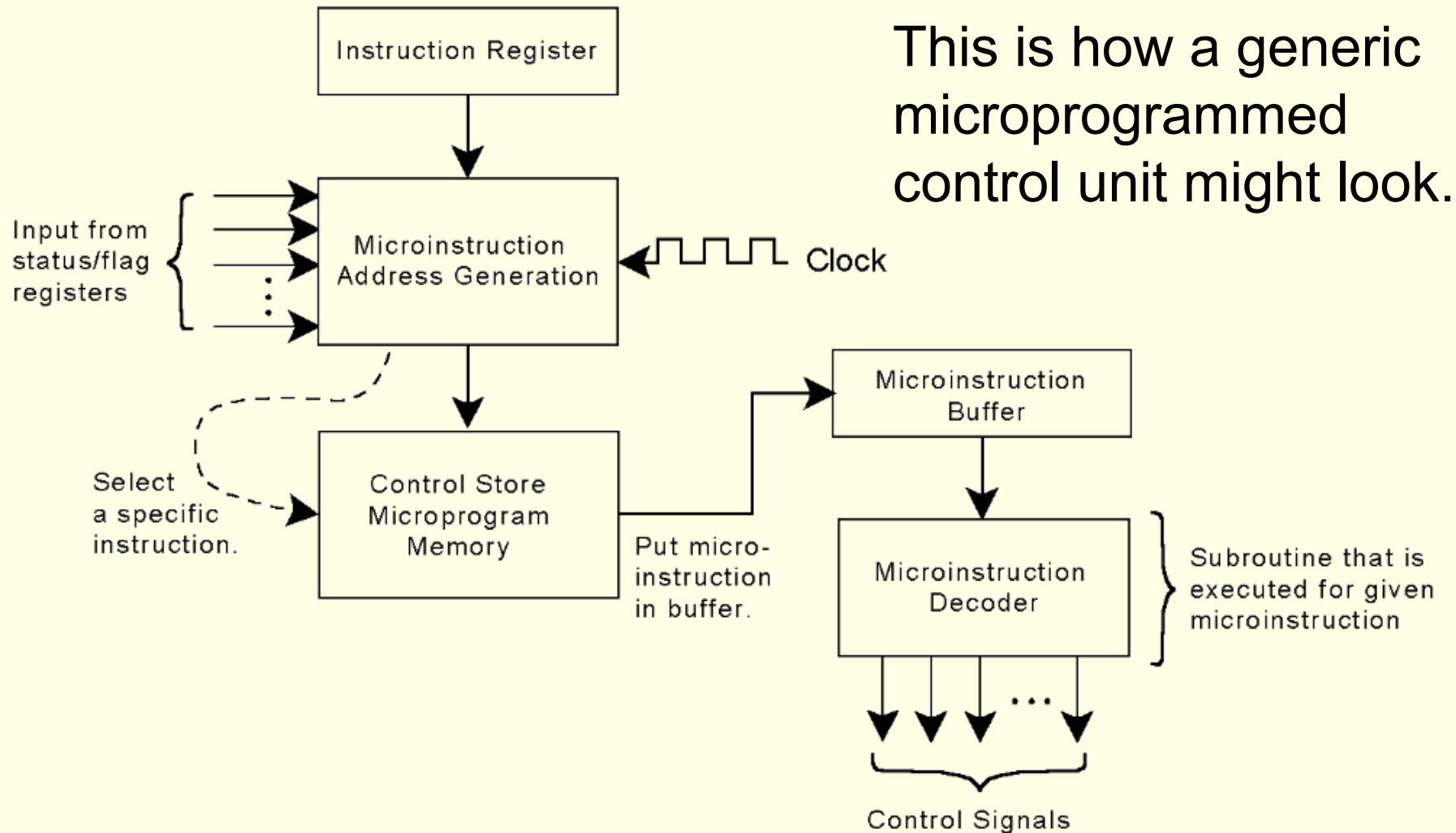


**Hardwired logic for
MARIE's Add =
0011 instruction.**

3.10.1 Instruction Decoding in MARIE

- In **microprogrammed control**, instruction microcode produces control signal changes.
- Machine instructions are the input for a microprogram that converts the 1s and 0s of an instruction into control signals.
- The microprogram is stored in firmware, which is also called the control store.
- A microcode instruction is retrieved during each clock cycle.

3.10.1 Instruction Decoding in MARIE

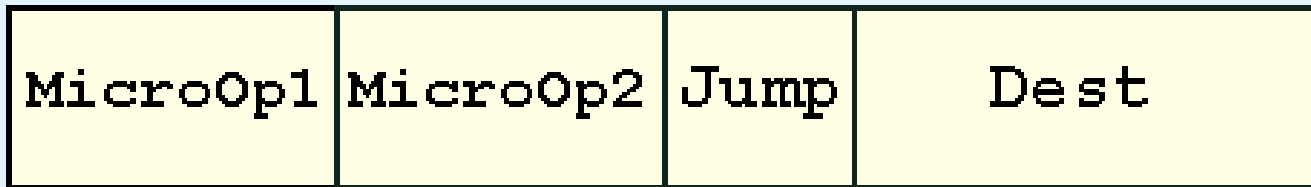


3.10.1 Instruction Decoding in MARIE

- Previously a list of the register transfer language steps for each of MARIE's instructions was provided.
- The microoperations given by each RTL define the operation of MARIE's control unit.
- Each microoperation consists of a distinctive signal pattern that is interpreted by the control unit and results in the execution of an instruction.

3.10.1 Instruction Decoding in MARIE

- If MARIE were microprogrammed, the microinstruction format might look like this:



17 13 12 8 7 6 0

- **MicroOp1** and **MicroOp2** : binary codes for 2 micro-operations (or a micro-operation + a parameter)
- **Jump** : a single bit indicating that the value in the **Dest** field is a valid address and should be placed in the micro-sequencer (i.e., the next micro-instruction to execute will be the one stored at address Dest)

3.10.1 Instruction Decoding in MARIE

- The table below contains MARIE's microoperation codes along with the corresponding RTL:

MicroOp Code	Microoperation	MicroOp Code	Microoperation
00000	NOP	01100	$MBR \leftarrow M[MAR]$
00001	$AC \leftarrow 0$	01101	$OutREG \leftarrow AC$
00010	$AC \leftarrow AC - MBR$	01110	$PC \leftarrow IR[11-0]$
00011	$AC \leftarrow AC + MBR$	01111	$PC \leftarrow MBR$
00100	$AC \leftarrow InREG$	10000	$PC \leftarrow PC + 1$
00101	$IR \leftarrow M[MAR]$	10001	If $AC = 00$
00110	$M[MAR] \leftarrow MBR$	10010	If $AC > 0$
00111	$MAR \leftarrow IR[11-0]$	10011	If $AC < 0$
01000	$MAR \leftarrow MBR$	10100	If $IR[11-10] = 00$
01001	$MAR \leftarrow PC$	10101	If $IR[11-10] = 01$
01010	$MAR \leftarrow X$	10110	If $IR[11-10] = 10$
01011	$MBR \leftarrow AC$	10111	If $IR[15-12] = MicroOp2[4-1]$

3.10.1 Instruction Decoding in MARIE

- First 9 lines of MARIE's microprogram (in RTL): first 4 are the fetch; the remaining ones implement the decode ...

Address	MicroOp 1	MicroOp 2	Jump	Dest
0000000	MAR $\leftarrow$ PC	NOP	0	0000000
0000001	IR $\leftarrow$ M[MAR]	NOP	0	0000000
0000010	PC $\leftarrow$ PC + 1	NOP	0	0000000
0000011	MAR $\leftarrow$ IR[11-0]	NOP	0	0000000
0000100	If IR[15-12] = MicroOp2[4-1]	00000	1	0100000
0000101	If IR[15-12] = MicroOp2[4-1]	00010	1	0100111
0000110	If IR[15-12] = MicroOp2[4-1]	00100	1	0101010
0000111	If IR[15-12] = MicroOp2[4-1]	00110	1	0101100
0001000	If IR[15-12] = MicroOp2[4-1]	01000	1	0101111
...	...	...	...	...

3.10.1 Instruction Decoding in MARIE

- **Add X** = {MAR $\leftarrow$ X; MBR $\leftarrow$ M[MAR]; AC $\leftarrow$ AC + MBR}
- An **Add** instruction is detected by the **MicroOp1** test at **Address** 0000111, once **MicroOp2** [4-1]=0011 (opcode of **Add**)
- Then, there's a jump (**Jump**=1) to **Address** 0101100 (once **Dest**=0101100), where begins the remaining micro-instructions of **Add** (**MAR** $\leftarrow$ **X** already happened at **MicroOp1** in **Address** 0000011)

Address	MicroOp1	MicroOp2	Jump	Dest
0101100	MBR $\leftarrow$ M[MAR]	NOP	0	0000000
0101100	AC $\leftarrow$ AC + MBR	NOP	1	0000000

- In the end of the last micro-instruction of **Add** , there is a jump (**Jump**=1) to the begining of the microprogram (**Dest**=0000000), which implies starting the fetch of the next MARIE instruction ...

3.10.1 Instruction Decoding in MARIE

- It's important to remember that a microprogrammed control unit works like a system-in-miniature.
- Microinstructions are fetched, decoded, and executed in the same manner as regular instructions.
- This extra level of instruction interpretation is what makes microprogrammed control slower than hardwired control.
- The advantages of microprogrammed control are that it can support very complicated instructions and only the microprogram needs to be changed if the instruction set changes (or an error is found).

3.11 Real World Architectures



- MARIE shares many features with modern architectures
 - but it is not an accurate depiction of them, and ...
 - without more registers or a stack, the use cases are very limited
- In the real-world, there is a plethora of architectures
- We focus only on two representatives of the two main architectural approaches: Intel (CISC-type) and MIPS (RISC-type)
 - **CISC** = *complex instruction set computer*: many different instructions, with variable size and some complex functions
 - **RISC** = *reduced instruction set computer*: small number of instructions, with fixed size; each instruction is relatively simple and executes fast; programs have more instructions

3.11 Real World Architectures



- The classic Intel architecture, the 8086, was born in 1979. It is a CISC architecture.
- It was adopted by IBM for its famed PC, which was released in 1981.
- The 8086 operated on 16-bit data words and supported 20-bit memory addresses.
- Later, to lower costs, the 8-bit 8088 was introduced. Like the 8086, it used 20-bit memory addresses.

What was the largest memory that the 8086 could address?

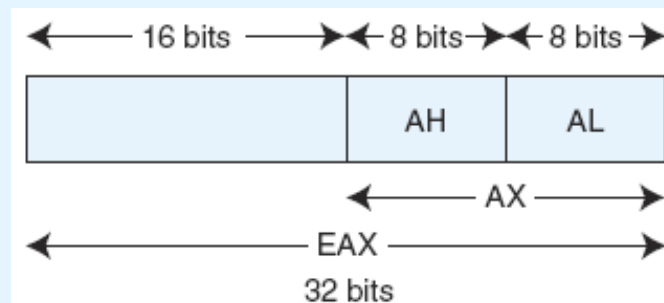
3.11 Real World Architectures



- The 8086 had four 16-bit general-purpose registers (AX = primary accumulator, BX = base register, CX = counter register, DS = data register) that could be accessed by the half-word (AH, BH, CH, DH vs AL, BL, CL, DL)
- It also had a flags register, an instruction register, an instruction pointer (IP) and a stack accessed through the base pointer (BP) and the stack pointer (SP).
- The 8086 had no built in floating-point processing.
- In 1980, Intel released the 8087 numeric coprocessor, but few users installed them because of their cost.

3.11 Real World Architectures

- In 1985, Intel introduced the 32-bit 80386.
 - it also had no built-in floating-point unit
- The 80486, introduced in 1989, was an 80386 with built-in floating-point processing and cache memory.
- Both the 80386 and 80486 offered downward compatibility with the 8086 and 8088.
 - software written for the smaller word systems was directed to use the lower 16 bits of the 32-bit registers.



3.11 Real World Architectures



- Much later (in 2000), Intel introduced the Pentium 4, an advanced 32-bit microprocessor
- It could run as fast as 3.8 GHz. This clock rate is nearly 800 times faster than the 4.77 MHz of the 8086.
- Speed enhancing features included multilevel cache and instruction pipelining.
- Intel, along with many others, married many of the ideas of RISC architectures with microprocessors that are largely CISC.

3.11 Real World Architectures



- Pentium: registos de 32 bits, data bus de 64 bits, super-escalar (várias ALUs, + de 1 instrução por ciclo)
- Pentium Pro: *branch-prediction* (previsão de saltos)
- Pentium II: instruções MMX (vetoriais) p/ multimédia
- Pentium III: suporte adicional para gráficos 3D (FPU)
- Pentium 4: nova microarquitetura (*netburst*), *pipelining* (encadeamento de instruções) de > profundidade, *caches* multinível, > velocidades de relógio (> 3GHz)
- Core : > desempenho, < consumo, *multicore*, 64 bits
- a Intel tem vindo a adotar ideias do mundo RISC:
 - instruções CISC são traduzidas em instruções RISC

3.11 Real World Architectures



- The MIPS CPUs (R3000, R4000, R5000, R8000, R10000) were one of the most successful in their class (RISC CPUs)
- In 1986 the first MIPS CPU was announced.
 - it was one of the 1st RISC processors
 - with 32-bit word size, could address 4GB of memory
 - only 55 instructions vs > 100 in a 8086
- MIPS was designed with performance in mind:
 - *load/store* architecture (only load and store instructions can access RAM); the large number of registers in the MIPS architecture keeps bus traffic to a minimum.
- Over the years, MIPS processors have been used in general purpose computers as well as in games.

3.11 Real World Architectures



- MIPS was designed with performance in mind: It is a *load/store* architecture, meaning that only the load and store instructions can access memory.
- The large number of registers in the MIPS architecture keeps bus traffic to a minimum.

How does this design affect performance?

Conclusion



- The major components of a computer system are its control unit, registers, memory, ALU, and data path.
- A built-in clock keeps everything synchronized.
- Control units can be microprogrammed or hardwired.
- Hardwired control units give better performance, while microprogrammed units are more adaptable.

Conclusion



- Computers run programs through iterative fetch-decode-execute cycles.
- Computers can only run programs that are in machine language.
- An assembler converts mnemonic code to machine language.
- The Intel architecture is a CISC architecture; MIPS is an example of a RISC architecture.

References

- book ECOA (6th Edition) – Chapter 4
(minus Memory Organization and Addressing)